

Product Requirements Document

Professional-Skepticism Engine — a Multi-Agent Audit Investigation Crew

Product	Professional-Skepticism Engine	Doc type	Product Requirements Document
Version	1.2	Status	Reviewed — CSV intake + per-agent I/O added
Owner	Capstone Team 01 (2 members)	Date	2026-06-26
Target release	5 working days	Pattern	Multi-agent (debate + verification)

What changed in v1.2. Case intake is now specified as an explicit **CSV ingestion + rule-based pre-filter pipeline** over a general-ledger extract (§9.1). Every agent's concrete **input and output contract** is referenced here and fully worked in the LLD and the Sample Output document, traced through **two cases** — one auto-cleared and one escalated.

1. Overview

The Professional-Skepticism Engine is a multi-agent audit investigation system. For each flagged transaction or piece of audit evidence, a crew of role-specialised Claude agents debates and verifies the case the way a real audit team does: an Evidence agent assembles the facts, a Challenger argues it is a problem, a Defender argues it is legitimate, an Adjudicator weighs both and assigns a risk + confidence + disposition, and a Verifier checks that the conclusion is grounded in evidence. Uncertain or material cases pause for a human reviewer; every decision and the full debate transcript are audited. The design directly operationalises professional skepticism (ISA 200) and the preparer → reviewer → partner review hierarchy.

2. Background & Problem Statement

A single LLM asked to judge a flagged item in one prompt is prone to confirmation bias and hallucination — it tends to find one plausible story and commit to it. In audit that is unacceptable: conclusions must be challenged, defended, and independently verified before they are relied upon. Rule-based detection is cheap and deterministic but cannot exercise judgement; a lone LLM exercises judgement but without challenge or verification. The product closes this gap with an adversarial, role-separated multi-agent design in which the independence of the agents is what produces a defensible verdict.

3. Why Multi-Agent Is Mandatory (not optional)

- **Independence is the mechanism.** The Challenger and Defender must reason separately; if one model played both roles it would rationalise a single narrative. Separation is what surfaces the counter-arguments.
- **Verification needs a different agent.** A model checking its own output for hallucination is unreliable; an independent Verifier with a distinct objective catches ungrounded claims and forces a revision.
- **Dynamic control.** The Supervisor re-plans and re-runs the debate when verification fails — a control loop, not a linear script.
- **Empirically testable.** §18 specifies an A/B evaluation (single-prompt vs. the crew) that measures the multi-agent uplift — proving the agents earn their place rather than asserting it.

Defensible claim: “We use multiple agents because the independent, adversarial reasoning measurably reduces false positives and catches hallucinations a single pass misses. Remove any agent and the verdict degrades — which we demonstrate empirically.”

4. Goals & Non-Goals

4.1 Goals

- **G1** — Produce a challenged, defended, and verified risk verdict for each case, with citations.
- **G2** — Demonstrate a measurable quality uplift of the multi-agent crew over a single-prompt baseline.
- **G3** — Keep a human durably in the loop for uncertain / material cases.
- **G4** — Make every verdict grounded (RAG policy + external data) and fully auditable, including the debate transcript.

4.2 Non-Goals (this release)

- Production ERP/SAP connectors; OCR; multi-tenant; RBAC/SSO at scale.
- Autonomous remediation / posting to the ledger; the engine recommends, humans decide.
- Large-scale fine-tuning; we use prompting + orchestration only.

5. Success Metrics (KPIs)

Metric	Definition	Target (MVP)
Multi-agent uplift	Crew vs. single-prompt: change in false-positive rate on a labelled set	≥ 20% FP reduction
Grounding rate	Adjudications passing the Verifier on first or second pass	≥ 95%
Hallucination catch	Ungrounded claims caught by the Verifier ÷ injected	≥ 80% on a seeded test
Auto-clear rate	High-confidence, verified, low-risk cases auto-cleared	30–50%
Reviewer effort	Minutes per case vs. manual baseline	≥ 50% reduction
Durability	Runs resumable after restart / mid-debate	100%

6. Personas & Stakeholders

Persona	Role / need
Audit Senior / Reviewer	Primary user; reviews the debate + disposition, approves / rejects with accountability.
Engagement Manager / Partner	Escalation approver for high-risk; owns sign-off.
Capstone Dev Team (2)	Build & operate; need modular, testable agents.
Methodology / Risk (future)	Owns the policy KB and the confidence / materiality thresholds.

7. User Stories

ID	User story
----	------------

US-1	As a reviewer, I want each case argued from both sides so I can trust the verdict isn't one-sided.
US-2	As a reviewer, I want the AI's conclusion verified against evidence so I'm not misled by a confident hallucination.
US-3	As a reviewer, I want to read the full debate transcript and approve / reject with a comment and signature.
US-4	As a partner, I want high-risk / material cases escalated to me automatically.
US-5	As compliance, I want an immutable trail of the debate, decision, and sources for every case.
US-6	As a developer, I want the run to survive a restart mid-investigation.
US-7	As an audit lead, I want to drop a GL export (CSV) and have the engine triage which rows become cases.

8. Scope

In scope (MVP, 5 days)	Out of scope (future)
GL CSV intake + rule pre-filter → synthetic audit cases as input	OCR, ERP/SAP connectors, live client data
6 role-specialised Claude agents + debate + verify loop	Many specialist agents; production fan-out at scale
Policy RAG over a small company-policy KB	Enterprise knowledge graph; auto policy ingestion
Confidence-gated human review (interrupt/resume)	RBAC/SSO, multi-tenant, e-signature platform
A/B eval vs single-prompt; SQLite + UI	Kubernetes, autoscaling, online learning

9. System Architecture & End-to-End Workflow

A LangGraph state machine orchestrates the crew. A deterministic **intake pipeline** turns a GL CSV into cases; the Supervisor routes; the Evidence agent grounds; the Challenger and Defender debate; the Adjudicator decides; the Verifier gates on grounding and can trigger a re-run. Diagram in Appendix A.

9.1 Case intake — CSV ingestion & rule-based pre-filter (Phase 0)

Input is a general-ledger extract such as `sample_gl_1000.csv` with columns: `txn_id`, `date`, `account`, `vendor`, `description`, `currency`, `amount`, `fx_rate_used`, `amount_usd`, `posted_by`. The engine does **not** send every row to the agent crew — that would be slow and costly. Instead a cheap, deterministic pre-filter decides which rows become investigation cases.

- Ingest & normalize** — each CSV row → a typed `Transaction` (Pydantic), with type/format validation.
- Rule-based pre-filter** — apply the flagging rules below; rows that trip ≥1 rule are flagged.
- Drop clean rows** — rows that trip no rule are recorded as cleared-at-intake and never reach the crew.
- Create case** — each flagged row → one case; seed `InvestigationState` (`flags[]`, `materiality`, `status=intake`) and checkpoint to SQLite with a `thread_id`.

Rule	Fields used	Why it flags
Materiality	<code>amount_usd ≥ 25000</code>	Mandatory-review threshold (ISA materiality).
FX conversion outlier	<code>currency ≠ USD</code> , <code>fx_rate_used</code>	Non-USD posting with an off-market rate = restatement / manipulation risk.
Round-number	<code>amount</code> ends in 000	Classic fabricated-amount red flag.

Segregation of duties	<code>posted_by</code> is generic/admin (e.g. <code>dadmin</code>)	Control weakness — preparer must ≠ approver.
Duplicate	same <code>vendor</code> + <code>amount_usd</code> + <code>date</code>	Double payment.
Unknown vendor	<code>vendor</code> not in vendor master	Related-party / shell-vendor risk.
Off-hours posting	<code>date</code> on a weekend / holiday	Unusual activity pattern.

9.2 End-to-end steps

#	Step	What happens / artifact
0	Case intake	GL CSV → ingest + normalize → rule pre-filter → flagged rows become cases (§9.1).
1	Supervisor · Planner	Decompose the case, decide which specialists to run, set escalation.
2	Evidence agent	RAG policy + history + live data → <code>evidence{}</code> with citations.
3a	Challenger	Argues fraud / error / control failure; cites red flags → <code>challenger_view</code> .
3b	Defender	Argues the legitimate business rationale → <code>defender_view</code> .
4	Adjudicator	Weights both → <code>adjudication{risk · confidence · disposition · citations}</code> .
5	Verifier · QA	Checks every claim is grounded; if not, returns to the Adjudicator / Supervisor to revise.
6	Confidence gate	≥0.90 & verified & Low → auto-clear · 0.70–0.90 → review · else manual.
7	Human review	<code>interrupt()</code> : reviewer sees the transcript, approves / rejects / signs → <code>resume()</code> .
8	Report & Audit	Risk-ranked case file (debate trail + citations) + immutable audit log.

10. Agent Specifications & I/O Contracts

Each agent is an independent Claude call with one objective and a typed input/output contract. The table below summarises what each consumes and produces; the concrete schemas live in the LLD §3–4, and both worked examples (auto-clear + escalated) are traced agent-by-agent in the **Sample Output** document.

Agent	Consumes (input)	Produces (output)
Supervisor · Planner	<code>case</code>	<code>plan{}</code> · routing · escalation decision
Evidence agent	<code>case</code> + plan	<code>evidence{}</code> + citations + <code>api_metadata</code>
Challenger	<code>case</code> + evidence	<code>challenger_view</code> (claims · red flags)
Defender	<code>case</code> + evidence (not the Challenger's reasoning)	<code>defender_view</code> (rationale · conditions)
Adjudicator	evidence + both views	<code>adjudication{risk · confidence · disposition · citations}</code>
Verifier · QA	adjudication + evidence	<code>verification{passed · issues · action}</code>
Confidence gate	adjudication + verification + materiality	<code>routing{auto · review · manual}</code>
Human reviewer	full bundle + transcript	<code>review_history[]</code> (action · comment · signature)

Report / Audit	everything	case file (MD/HTML/JSON) + <code>audit_history[]</code>
----------------	------------	---

11. Functional Requirements

MUST **SHOULD** **COULD** priorities.

11.1 Intake, Orchestration & Evidence

ID	Requirement	Priority
FR-IN-1	Ingest a GL CSV, normalize each row to a typed Transaction, and run the rule pre-filter; only flagged rows become cases.	MUST
FR-IN-2	Record clean (unflagged) rows as cleared-at-intake; never send them to the crew.	MUST
FR-OR-1	The Supervisor must orchestrate the crew over a typed, checkpointed state and route per case.	MUST
FR-OR-2	On Verifier failure, the Supervisor re-runs the debate / adjudication up to 2 rounds; if it still fails, the case auto-escalates.	MUST
FR-EV-1	The Evidence agent must assemble an evidence pack from Policy RAG + transaction history + ≥ 1 external source, with citations.	MUST
FR-EV-2	External calls must time out and fall back to cache; missing evidence is recorded, not fatal.	MUST

11.2 Debate, Adjudication & Verification

ID	Requirement	Priority
FR-CH-1	The Challenger must produce a structured case-for-concern (claims + cited red flags) from the evidence only.	MUST
FR-DF-1	The Defender must produce a structured case-for-legitimacy independently of the Challenger.	MUST
FR-AD-1	The Adjudicator must weigh both views and output risk_level + confidence + disposition + citations.	MUST
FR-VE-1	The Verifier must check each adjudicator claim is supported by retrieved evidence and flag unsupported ones.	MUST
FR-VE-2	If verification fails, the case must loop back for revision (bounded), not pass silently.	MUST

11.3 Decision, Human review, Report & Audit

ID	Requirement	Priority
FR-CG-1	Route by confidence + verification + materiality: auto-clear / reviewer / manual.	MUST
FR-HR-1	Review / manual cases pause via <code>interrupt()</code> with state + transcript persisted.	MUST
FR-HR-3	Material / high-risk cases auto-escalate to a partner; SoD enforced (preparer $\neq$ approver).	SHOULD
FR-RP-1	Produce a risk-ranked case file (debate trail + citations) in Markdown / HTML.	MUST
FR-AU-1	Append-only, hash-chained audit log of the debate, decision, reviewer, and sources.	MUST

12. Decision & Confidence Logic

Condition	Route
confidence $\geq$ 0.90 AND Verifier passed AND risk = Low AND amount < materiality	Auto-clear (QA-sampled)
$0.70 \leq$ confidence < 0.90 (and verified)	Reviewer queue
confidence < 0.70 OR Verifier failed twice OR risk = High OR amount $\geq$ materiality	Manual / partner escalation

Materiality threshold = **USD 25,000**. Mandatory partner review is triggered by any of: amount $\geq$ USD 25,000, High risk, or failed verification. Thresholds (0.90 / 0.70) and materiality are configuration owned by Audit Methodology / Risk.

13. Data Model — InvestigationState

```
InvestigationState = {  
  case · plan · evidence{} · debate_transcript[] ·  
  challenger_view · defender_view ·  
  adjudication{risk·confidence·disposition·citations} ·  
  verification{passed·issues·action} ·  
  routing · review_history[] · audit_history[] · round · report  
}
```

14. Technology Stack & External APIs

Layer	Technology	Purpose
Intake	pandas / csv + Pydantic v2	Ingest GL CSV, normalize, rule pre-filter
Orchestration	LangGraph (StateGraph + SQLite checkpointer, interrupt/resume)	Durable multi-agent control loop
LLM	Anthropic Claude via langchain-anthropic	Sonnet for the 6 reasoning agents; Haiku for report generation
Schemas	Pydantic v2	Typed state + per-agent structured outputs
RAG / vector DB	Chroma or FAISS + embeddings	Policy-clause retrieval (hybrid search)
Real-time HTTP	httpx	Evidence agent external calls (FX)
Persistence	SQLite	Checkpoints, debate transcript, audit log
UI	Next.js / React (or Streamlit)	Case list, debate viewer, review queue

15. Evaluation Plan (proving the multi-agent uplift)

- Build a small labelled set (30–50 cases with known fraud / legitimate ground truth) from the synthetic GL data.
- Run each case through (A) a single Claude prompt baseline and (B) the multi-agent crew.
- Compare false-positive rate, false-negative rate, and grounding (citation) quality.
- Inject ungrounded claims to measure the Verifier's hallucination-catch rate.

- Report the delta — the headline evidence that the multi-agent design is necessary, not decorative.

16. Acceptance Criteria (Definition of Done)

- A GL CSV is ingested; the pre-filter creates cases only for flagged rows and drops clean rows.
- A case runs end-to-end: evidence → Challenger & Defender debate → Adjudicator → Verifier → verdict.
- The Verifier demonstrably sends at least one ungrounded case back for revision.
- At least one case auto-clears and one routes to a human, who approves / rejects with a signed comment; the graph resumes.
- The A/B eval shows a measurable improvement of the crew over the single-prompt baseline.
- A run survives a restart mid-debate; pytest passes; docker run works.

Appendix A — Architecture Diagram

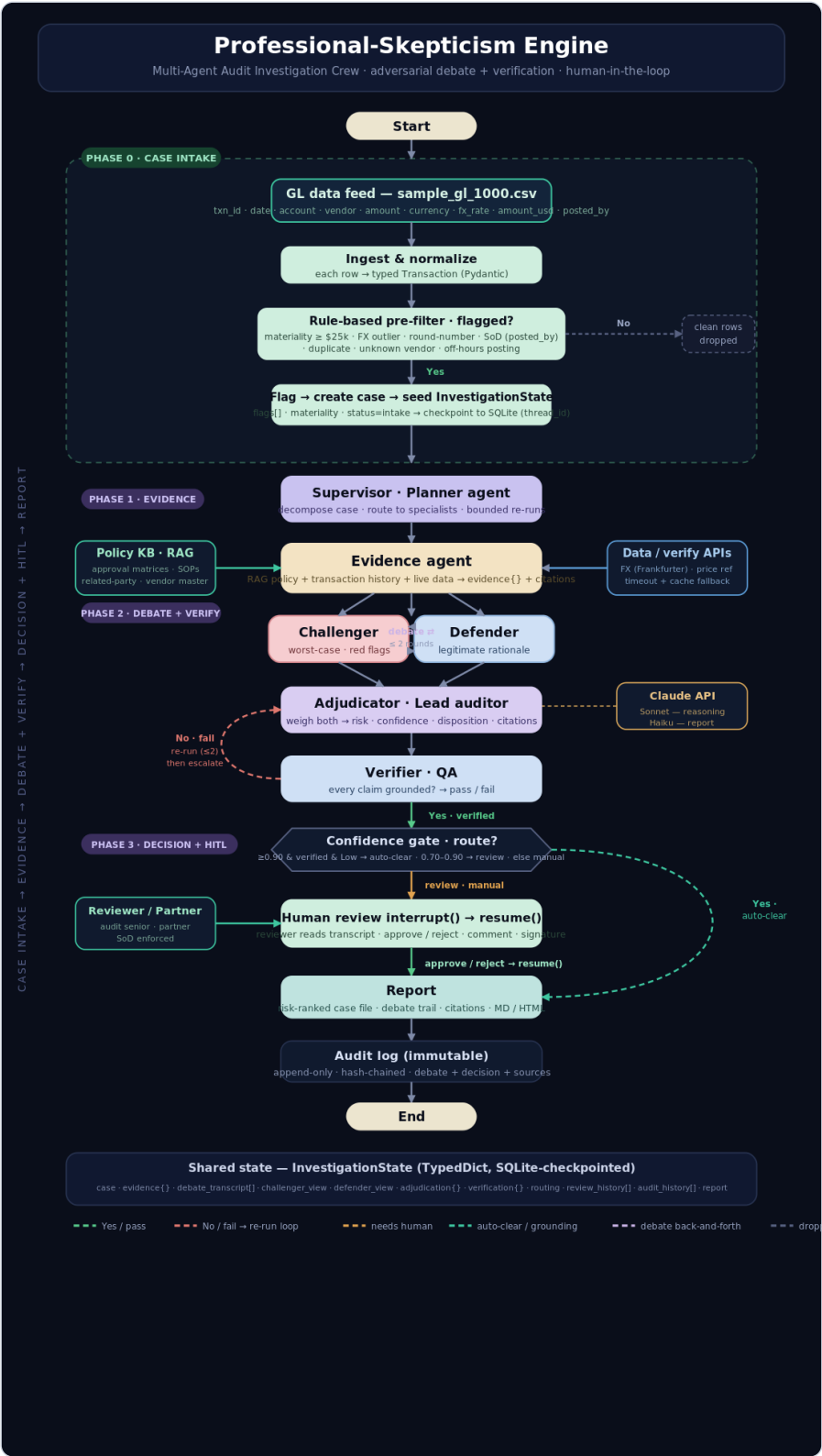


Figure 1 — The investigation crew. A GL-CSV intake pipeline pre-filters rows into cases; Evidence feeds an adversarial Challenger/Defender debate; the Adjudicator decides; the Verifier gates on grounding and loops back on failure; the confidence gate routes to auto-clear or the human-review interrupt. Every agent is a role-specialised Claude call; SQLite persists state + transcript. Yes/No branches and the re-run loops are labelled on the diagram.